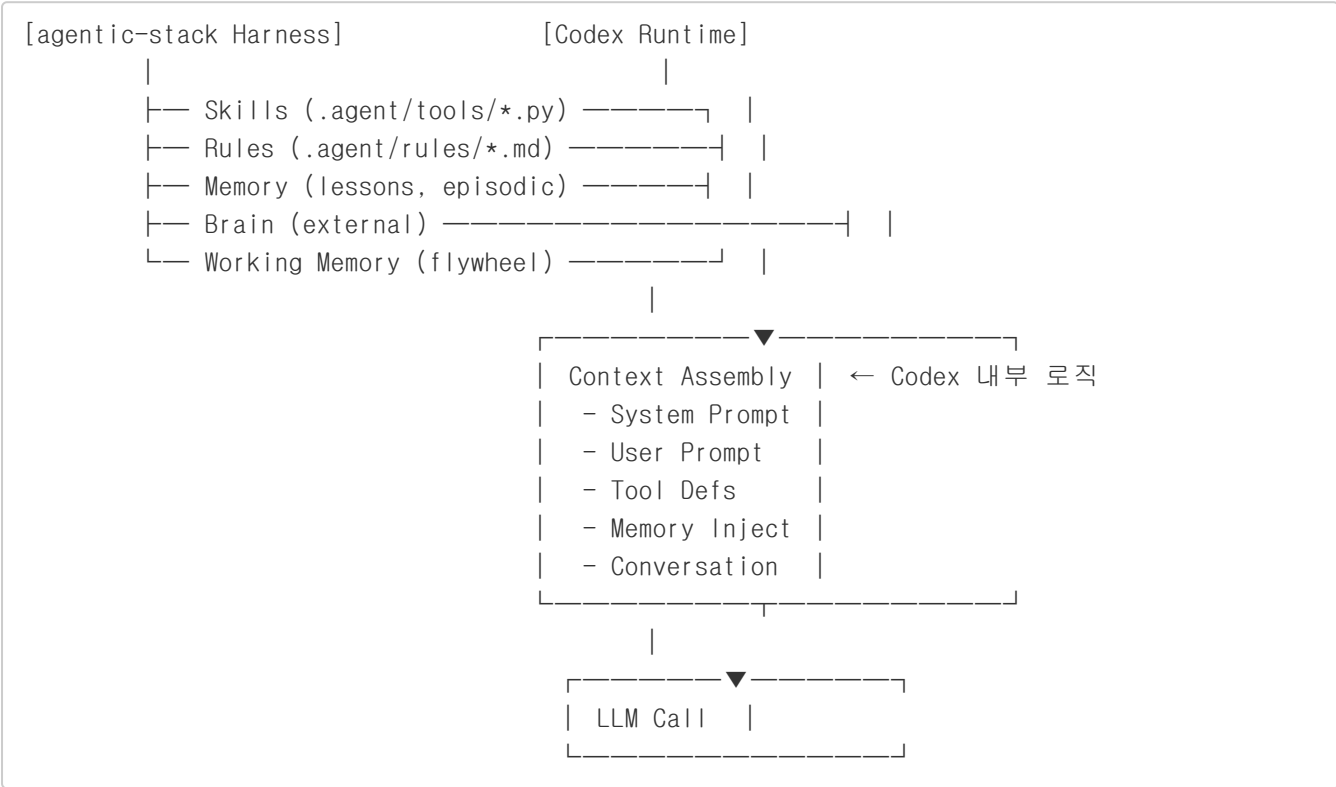


네, 정확히 그 지점이 "Agent의 자율성과 개발자의 통제권이 충돌하는 곳"입니다. 아래에서 agentic-stack의 실제 코드 구조를 바탕으로, Codex의 Context Assembly Pipeline을 낱낱이 분석해드리겠습니다.

1. 전체 Context Injection 파이프라인: 주체와 흐름

1.1 핵심 답변: Codex가 결정하지만, Harness가 그 재료를 통제한다

LLM 호출 시 최종 Context를 조립하는 주체는 Codex(또는 Cursor, Windsurf 등)입니다. 하지만 그 재료가 되는 Agent Memory, Rules, Skills, Lessons을 결정하고 공급하는 것은 `agentic-stack`의 Harness입니다. 즉, Harness는 '재료'를 통제하고, Codex는 '레시피'를 통제합니다.



1.2 Codex가 Context를 조립하는 6단계 (실제 내부 구조)

Codex의 Context Assembly는 보통 다음과 같은 계층으로 이루어집니다:

계층	내용	결정 주체	Harness 개입 가능성
1. System Prompt (Fixed)	Agent의 역할, 기본 지침, 도구 목록 선언	Codex (하드코딩)	없음
2. Harness Injects	Skills, Rules, Accepted Lessons, Working Memory	Harness (agentic-stack)	직접 통제
3. Conversation History	이전 턴의 user/assistant 메시지	Codex (버퍼 관리)	간접 (history pruning)
4. Flywheel Traces	이전 실행의 성공/실패 트레이스	Codex (선택적)	"stay out" 기본
5. Runtime Indexes	현재 코드베이스 색인, LSP 정보	Codex (실시간)	없음
6. User Prompt	현재 턴의 사용자 입력	사용자	직접

핵심: `Full memory intent` 는 이 6단계 중 2번 계층(Harness Injects) 에서 무엇을 포함할지에 대한 정책을 바꾼 것입니다.

2. 각 Harness Mechanism이 Context에 미치는 구체적 영향

2.1 Full Memory Intent

```
"preferences, accepted lessons, skills, working memory, episodic/history logs,
and candidate lessons" → IN
"flywheel traces, runtime indexes, and caches" → OUT
```

Context에 주입되는 방식

LLM 호출 시, System Prompt 뒤에 다음과 같은 섹션이 자동 주입됩니다:

```
## Your Memory (HARNESS INJECTED)
### Accepted Lessons
- [lesson_1] When writing SQL queries, always use parameterized queries. (accepted 2026-06-15)
- [lesson_2] For large CSV files, use DuckDB instead of Pandas. (accepted 2026-06-20)

### Working Memory
- Current task: Building OKF bundle for 2026-1 semester projects
- Last action: Created semester/2026-1/index.md

### Episodic Logs (Recent 5)
- [2026-07-01 14:23] User asked for student count → planner generated IR with JOIN
- [2026-07-01 14:30] Executor ran successfully → 42 students returned

### Candidate Lessons (for your consideration)
- [candidate] Use read_index before search_hierarchy for faster traversal (2 occurrences)
```

영향 분석

- **Token 사용량:** 크게 증가 (과거에는 없었던 메모리 섹션이 추가됨)
- **일관성:** Accepted Lessons 덕분에 Agent가 같은 실수를 반복하지 않음
- **오염 위험:** Episodic Logs가 너무 길면 오래된 컨텍스트가 현재 질문과 무관하게 주입되어 혼란 유발

2.2 Semantic Lesson Retraction

```
# .agent/tools/retract_lesson.py
def retract_lesson(lesson_id: str, reason: str) -> dict:
    """
    Append-only audit log에 retraction 이벤트를 기록하고,
    이후 Full Memory Intent에서 해당 lesson을 제외하도록 마킹.
    """
```

Context에 주입되는 방식

Retraction 이후, Accepted Lessons 섹션에서 해당 lesson이 사라집니다.
대신, audit log에는 "retracted" 이벤트가 남아 감사 추적이 가능합니다.

```
## Your Memory
### Accepted Lessons
```

```
- [lesson_1] ... (active)
- [lesson_2] ... RETRACTED (reason: "Outdated after DuckDB 1.0 release") ← 제외됨
```

영향 분석

- **Context 오염 방지:** 오래된 지식이 영구적으로 주입되는 것을 막음
- **Append-only 무결성:** 삭제가 아니라 '마킹'이므로, 필요 시 복원 가능
- **Audit 가능:** 누가, 언제, 왜 retract했는지 추적 가능

2.3 Brain (External Memory Bridge)

```
agentic-stack brain onboard --agents codex,cursor --yes
```

이것은 `brain_bridge.py` 를 설치하여 **cross-harness**, **cross-agent** 메모리를 가능하게 합니다.

Context에 주입되는 방식

Codex가 LLM 호출 전에 Brain에 쿼리하도록 도구가 추가됩니다.

```
# .agent/tools/brain_bridge.py (skeleton)
def brain_recall(query: str, limit: int = 3) -> list[dict]:
    """Query the external Brain for relevant long-term memory."""
    # brain CLI를 호출하여 임베딩 검색
```

이 도구가 등록되면, Agent는 **필요할 때 능동적으로** Brain을 조회할 수 있습니다.
즉, 수동적 주입이 아니라 **도구 호출을 통한 능동적 회상**이 됩니다.

Context에 미치는 영향

- **Token 절약:** Brain이 컨텍스트에 바로 주입되지 않고, 필요 시 도구 호출로 가져옴
- **Cross-Project:** 다른 프로젝트에서의 학습이 현재 프로젝트에 공유됨
- **지연 시간:** 매 질문마다 Brain을 조회하면 latency 증가 (도구 호출 1회 추가)

3. CKOS/KnowledgeChef 개발자를 위한 실전 가이드

3.1 당신이 통제할 수 있는 것 (Harness 수준)

이러한 분석을 바탕으로, CKOS의 컨텍스트 품질을 관리하는 규칙을 정할 수 있습니다.

A. `AGENTS.md` 에 명시할 Context Policy

```
# Context Injection Policy for CKOS Planner Agent

## Memory Intent
- Always inject "Accepted Lessons" (max 10)
- Always inject "Working Memory" (current task only)
- Do NOT inject "Episodic Logs" during Planning phase (only Executor needs it)
- Do NOT inject "Candidate Lessons" (human review required)

## Brain Recall Trigger
- Planner may call `brain_recall` ONLY when:
  1. The current question involves a domain NOT in the OKF bundle
  2. A previously retracted lesson might apply
- Executor may call `brain_recall` for code patterns only.
```

```
## Lesson Lifecycle
- New lesson is "candidate" for 3 successful uses
- Then it becomes "accepted" (auto-injected)
- If a lesson causes 2 consecutive failures, it is retracted (audited)
```

B. Context Debugging Tool

당신의 Agent에게 다음과 같은 디버그 도구를 추가하세요.

```
# .agent/tools/debug_context.py
def show_injected_context() -> dict:
    """
    현재 LLM 호출에 주입된 모든 컨텍스트를 반환.
    (Harness가 로깅한 내용 기반)
    """
    return {
        "accepted_lessons": [...], # 몇 개, 총 토큰 수
        "working_memory": {...},
        "episodic_logs_count": 5,
        "total_injected_tokens": 1234,
        "brain_queries_this_session": 2,
    }
```

3.2 Codex의 내부 로직에 의존하지 않는 설계 원칙

Codex의 **Context Assembly**는 블랙박스입니다.

따라서 CKOS의 Planner와 Executor는 다음을 지켜야 합니다.

1.

Skill과 Rule은 반드시 `.agent/tools/`에 독립적인 Python 함수로 존재할 것

→ Codex가 도구 정의를 바꾸거나 생각해두, Executor는 항상 동일한 함수를 호출함.

2.

Planner의 계획은 Knowledge IR이라는 중립 형식으로 출력할 것

→ Codex가 Planner의 내부 프롬프트를 어떻게 조립하든, 최종 출력은 IR JSON으로 강제함.

3.

컨텍스트 오염을 감지하는 Watchdog 도구

```
def check_context_hygiene(max_lessons: int = 10) -> bool:
    """주입된 lessons이 너무 많거나, irrelevant한 것이 있는지 검사"""
```

4.

Brain은 "전역 기억"으로만 사용하고, 프로젝트 로컬 지식은 OKF Bundle에 저장

→ Brain에 OKF 데이터를 중복 저장하면, 컨텍스트 일관성이 깨짐.

3.3 핵심 통찰: Harness가 컨텍스트의 '정보 엔트로피'를 통제한다

agentic-stack의 모든 신규 기능은 결국 LLM이 보는 컨텍스트의 품질을 통제하는 것입니다.

- 너무 많은 메모리 → 혼란, 토큰 낭비
- 너무 적은 메모리 → 같은 실수 반복
- 오염된 메모리 → 잘못된 추론

CKOS 개발자는 이 균형을 **자동으로 조절**하는 메타 규칙을 설계해야 합니다.

이것이 바로 이전에 논의한 **Meta-Compiler**의 역할입니다.

요약하면, Codex의 Context Assembly는 당신이 만든 Harness 재료를 레시피에 따라 조리하는 주방장입니다.

당신은 재료(메모리, 룰, 스킬)의 신선도와 양을 통제할 수 있습니다.

이 통제권을 자동화하는 시스템이 바로 **Cognitive Knowledge OS**의 궁극적 목표입니다.

1. agentic-stack이 전용 Agent 구현에 도움이 될까?

결론부터 말씀드리면, agentic-stack은 전용 Agent의 '빠대'와 '지식/메모리 관리' 측면에서 매우 유용하지만, Multi-user 서비스로 확장하려면 상당한 추가 개발이 필요합니다.

✅ agentic-stack의 강점 (활용 가능한 부분)

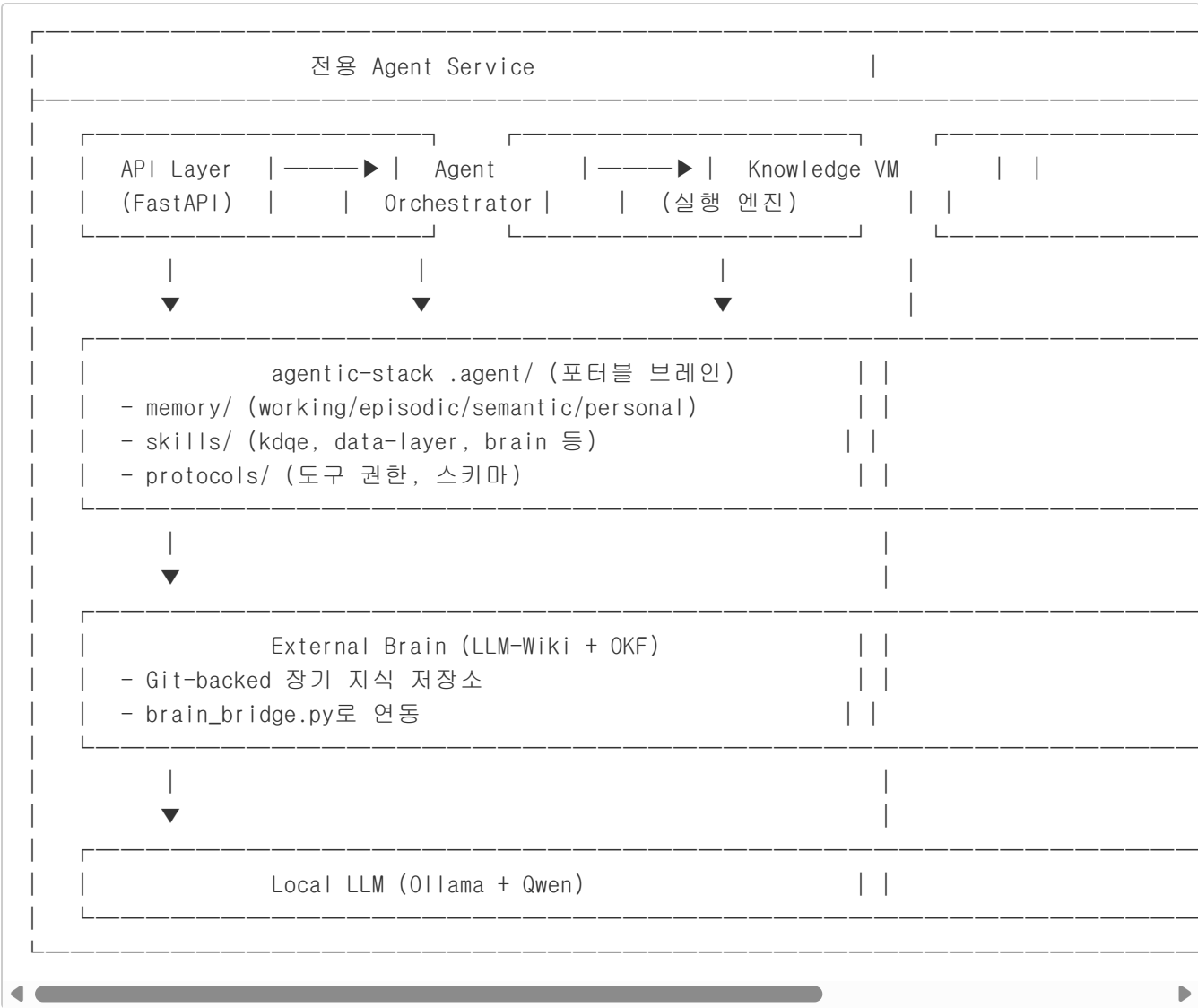
기능	설명	전용 Agent에서의 활용
포터블 <code>.agent/</code> 브레인	메모리(working/episodic/semantic/personal), 스킬, 프로토콜을 하나의 폴더에 담아 다양한 코딩 에이전트(Claude Code, Cursor, Hermes, Codex 등)와 공유	지식 베이스와 에이전트 설정을 표준화된 구조로 관리할 수 있어, 로컬 LLM 기반 에이전트도 동일한 인터페이스로 지식을 읽고 쓸 수 있음
외부 Brain 통합 (v0.18.0)	<code>codejunkie99/brain</code> 이라는 Git-backed 장기 메모리 CLI/TUI/MCP 서버와 브릿지 제공	LLM-Wiki를 External Brain으로 연결하면, 에이전트가 프로젝트 간 지식을 Recall하고, 장기 기억을 유지할 수 있음
Data Layer (모니터링)	여러 에이전트의 활동, 토큰/비용 추정, KPI 요약, 일일 대시보드를 로컬에서 제공	전용 Agent 서비스의 운영 모니터링, 사용량 추적, 비용 관리에 그대로 활용 가능
Flywheel (자기 진화)	승인된 실행 기록을 Trace, Eval Case, Training-ready JSONL 등으로 변환	에이전트의 자가 진단 및 지속적 학습 파이프라인 구축에 활용 가능
다양한 Harness 지원	Claude Code, Cursor, Hermes, Codex, Gemini CLI, DIY Python 등과 연결 가능	Coding Agent 단계에서는 원하는 도구로 자유롭게 테스트하고, 이후 전용 Agent로 전환할 때도 <code>.agent/</code> 구조를 재사용 가능

⚠️ agentic-stack의 한계 (직접 개발 필요한 부분)

한계	설명	전용 Agent 개발 시 대응 방안
Multi-user 미지원	agentic-stack은 단일 사용자/단일 프로젝트 환경을 가정	사용자 인증, 멀티테넌시, 세션 격리, 권한 관리 등을 직접 구현해야 함
로컬 LLM 연결 부재	agentic-stack 자체는 LLM을 내장하지 않고, Harness(Codex 등)에 위임	Ollama, vLLM, Llama.cpp 등 로컬 LLM 서버와 연동하는 Adapter/API를 직접 개발
서비스형 API 미제공	CLI/TUI 기반 도구로, REST API나 WebSocket 서버를 제공하지 않음	FastAPI, Flask 등으로 HTTP API 서버를 구축하고, agentic-stack의 내부 함수를 호출하는 Wrapper 개발 필요
사용자별 데이터 격리	<code>.agent/</code> 폴더가 프로젝트 단위로 단일 저장소를 사용	사용자/조직별로 별도의 <code>.agent/</code> 브랜치나 디렉토리를 분리하고, DB로 메타데이터를 관리해야 함

2. Coding Agent → 전용 Agent Service 전환을 위한 개발 가이드

🏗️ 전체 아키텍처 제안



단계별 개발 로드맵

Phase 1: agentic-stack 기반 Coding Agent 고도화 (현재 단계)

작업	설명	비고
LLM-Wiki → External Brain 연결	<code>agentic-stack brain</code> 명령어로 Brain 상태 확인, 프로젝트 온보딩, 글로벌 메모리 검색, durable note 작성 등 가능	<code>.agent/tools/brain_bridge.py</code> 를 활용해 Host Agent가 Cross-project Recall을 수행하도록 구성
Brain Seed Skill 활성화	Brain Skill이 에이전트에게 Brain 메모리 Query/Write 시점을 알려주고, Secret handling을 명시적으로 관리	Coding Agent가 지식 추가/검색/자가진단을 자동으로 수행하도록 Skill 내장
Data Layer로 모니터링	<code>agentic-stack doctor</code> , <code>data-layer</code> 스킬로 Agent 활동, 토큰/비용, KPI를 대시보드화	Coding Agent 실행 로그를 수집하고, Flywheel Artifact(Trace, Eval Case)를 생성하여 자기 진화 기반 마련

Phase 2: 전용 Agent Service 코어 개발

작업	설명	구현 포인트
로컬 LLM Adapter 구현	Ollama, vLLM 등 OpenAI-compatible API를 호출하는 Wrapper	<code>langchain_community.llms.Ollama</code> 또는 <code>openai</code> 클라이언트로 Qwen/Gemma 등 연결

작업	설명	구현 포인트
API 서버 구축	FastAPI로 <code>/query</code> , <code>/add_knowledge</code> , <code>/search</code> , <code>/diagnose</code> 등 엔드포인트 제공	agentic-stack의 <code>plan()</code> 함수를 호출하고, 결과를 JSON으로 반환
Multi-user 인증/세션	JWT 또는 API Key 기반 사용자 인증, 사용자별 <code>.agent/</code> 브랜치 격리	각 <code>user_id</code> 에 대해 별도의 <code>.agent/</code> 디렉토리(또는 Git 브랜치)를 생성하고, <code>system_model</code> 을 동적으로 로드
지식 추가/검색/자가진단 API	LLM-Wiki(OKF)에 새로운 Concept/Data를 추가하고, 검색하며, 무결성을 진단하는 엔드포인트	<code>kchef add</code> , <code>kchef validate</code> , <code>brain_bridge.py</code> 를 호출하는 Service Layer 구현

Phase 3: 확장 및 최적화

작업	설명
멀티테넌시 데이터 격리	사용자/조직별 OKF 번들, SQLite DB, Vector DB를 분리
캐싱 및 성능 최적화	자주 조회되는 Concept, IR 결과를 Redis 등에 캐싱
Webhook/Event 기반 자가진화	에이전트 실행 완료 시 Flywheel Artifact를 자동 생성하고, Brain에 학습 피드백을 기록
관리자 대시보드	사용자별 사용량, 토큰 소비, 에이전트 활동 로그를 시각화

🔧 핵심 구현 예시 (Python)

```
# agent_service/api.py
from fastapi import FastAPI, Depends, HTTPException
from kchef.planner.pipeline import PlannerPipeline, load_system_model
from kchef.planner.simulator import MentalSimulator
from langchain_community.llms import Ollama

app = FastAPI()

# 사용자별 .agent/ 경로 매핑
USER_AGENT_PATHS = {
    "user1": "/path/to/user1/.agent",
    "user2": "/path/to/user2/.agent",
}

def get_planner(user_id: str):
    agent_path = USER_AGENT_PATHS.get(user_id)
    if not agent_path:
        raise HTTPException(404, "User not found")
    system_model = load_system_model(bundle_path=f"{agent_path}/wiki")
    return PlannerPipeline(system_model, skills_dir=f"{agent_path}/skills")

@app.post("/query")
def query(question: str, user_id: str):
    planner = get_planner(user_id)
    ir = planner.plan(question)
    # 로컬 LLM으로 최종 응답 생성
    llm = Ollama(model="qwen2.5:7b")
```

```
response = llm.invoke(f"Based on this plan: {ir.dict()}, answer: {question}")
return {"answer": response, "ir": ir.dict()}
```

🔴 결론

agentic-stack은 포터블 브레인(.agent/) 과 External Brain(brain_bridge) 을 통해 지식과 메모리를 표준화하고, Data Layer와 Flywheel로 모니터링과 자기 진화의 기반을 제공합니다. 이는 전용 Agent Service의 '**지식 관리 및 메모리 계층**'을 단기간에 구축할 수 있게 해주는 강력한 자산입니다.

다만, Multi-user 지원, 로컬 LLM 연동, REST API 서버는 직접 개발해야 하며, agentic-stack은 이 부분을 대체하지 않고 '**내부 엔진**'으로 활용하는 것이 바람직합니다.

💡 **권장 전략:** Coding Agent 단계에서는 agentic-stack의 모든 기능을 활용해 지식베이스와 에이전트 워크플로를 완성하고, 전용 Agent Service로 전환할 때는 agentic-stack의 .agent/ 폴더와 brain_bridge를 그대로 재사용하면서, 상위에 Multi-user API와 로컬 LLM Adapter를 씌우는 **Adapter 패턴**을 적용하세요.

우리가 지금까지 논의한 Cognitive Compiler / CKOS는 본질적으로 **도메인 특화 마이크로-에이전트 아키텍처**입니다. 전통적인 마이크로서비스와 거의 동일한 구조를 가지고 있으며, 그 차이는 "서비스 내부 로직이 LLM으로 대체되었다"는 점과 "메시지가 자연어를 포함한 구조화된 지식 표현"이라는 점뿐입니다.

1. Micro-Service ↔ Agent Architecture 완전 대응

전통적 Micro-Service	CKOS Agent System	비고
Actor (Service)	Agent (Planner, Executor, MemoryCurator)	독립된 역할, 각자의 상태 보유
Message Passing (gRPC, RabbitMQ, HTTP JSON)	Message (Pydantic Model / JSON) - PlanRequest, KnowledgeIR, ExecutionResult 등	정해진 계약(Contract)에 따라 주고받는 구조화 데이터
In-Memory DB (Redis, session cache)	Working Memory (현재 작업 상태, conversation history)	휘발성, 빠른 접근
Persistent DB (PostgreSQL, file system)	.agent/ 폴더 - SKILL.md, RULE.md, accepted lessons, tools	장기 기억, 코드/규칙 저장소
Shared External DB (Data Lake, cross-service cache)	Brain (external vector DB / cross-harness memory)	에이전트 간, 프로젝트 간 공유 지식
API Gateway / Router	Query Router Agent (사용자 질문을 분류하여 적절한 Planner에게 전달)	진입점

따라서, Agents의 Message는 JSON data입니다.

하지만 단순한 JSON이 아니라 **도메인 의미**를 담은 Knowledge IR, ExecutionResult, MemoryUpdate 같은 **도메인 객체**입니다.

2. Domain-Specific Micro-Agent 구조 설계 (OKF 기반)

이 아키텍처는 '**OKF 지식 공간을 대상으로 사용자 질문에 답변하는 도메인 특화 시스템**'을 위해 최적화되어 있습니다.

2.1 에이전트 목록 및 역할

Agent	역할	보유 상태 (자체 DB)
QueryRouter	사용자 질문을 분석하여 목적(Goal)을 분류하고, 적합한 Planner에게 전달	Conversation History
Planner	Knowledge Space(index.md)를 탐색하여 Knowledge IR(실행 계획) 생성	탐색 히스토리, Lessons (계획 패턴)
Executor	Knowledge IR을 받아 실제 데이터(CSV, 문서, API)에 접근하여 결과 생성	데이터 접근 통계, 오류 로그
MemoryCurator	성공/실패 경험을 분석하여 Lesson을 추출·정리·진화시킴	Lessons, Rules, SKILL.md
BrainGateway	외부 Brain에 접근하여 다른 프로젝트의 지식을 검색하거나, 현재 지식을 저장	Brain 연결 정보

2.2 Message Definitions (Pydantic Models)

```
# messages.py
from pydantic import BaseModel, Field
from typing import Literal, Optional, List, Dict, Any
import uuid

class AgentMessage(BaseModel):
    msg_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    sender: str # agent name
    receiver: str # agent name
    timestamp: float

# Router → Planner
class PlanRequest(AgentMessage):
    question: str
    user_context: Optional[Dict[str, Any]] = None # 세션, 권한 등

# Planner → Executor
class KnowledgeIR(AgentMessage):
    goal: Literal["QUERY", "AGGREGATE", "SUMMARIZE", "COMPARE"]
    steps: List[Dict[str, Any]] # DataStep 또는 DocumentStep 직렬화
    output_structure: Dict[str, str]

# Executor → Router (최종 응답)
class ExecutionResult(AgentMessage):
    result: Any
    citations: List[str]
    summary: str

# Planner → MemoryCurator
class ExperienceRecord(AgentMessage):
    question: str
    plan: KnowledgeIR
    success: bool
    error_detail: Optional[str] = None

# MemoryCurator → Planner (업데이트 알림)
class MemoryUpdate(AgentMessage):
```

```

new_lessons: List[Dict[str, str]]
updated_skills: Optional[str] = None

# Brain Gateway
class BrainQuery(AgentMessage):
    query_text: str
    max_results: int = 5

class BrainStore(AgentMessage):
    content: str
    metadata: Dict[str, Any]

```

2.3 에이전트 간 메시지 흐름 (시나리오: 사용자 질문 "2026-1학기 VIP 고객 주문 총액")

```

[User]
|
▼
[QueryRouter]
| PlanRequest(question, user_context)
▼
[Planner]
| 1. OKF index.md 탐색 (스키마 로드)
| 2. KnowledgeIR 생성 (steps에 데이터 소스, 조인, 집계)
| 3. MemoryCurator에 이전 유사 계획 질의 (BrainGateway 경유 가능)
| KnowledgeIR 전송
▼
[Executor]
| 1. IR 해석 → SQL 또는 Pandas 연산
| 2. 실제 CSV/DB 액세스 (로컬 파일, MCP 도구 등)
| 3. 결과 생성
| ExecutionResult 전송
▼
[QueryRouter]
| 최종 응답을 사용자에게 반환
▼
[User]

```

병렬로:

```

[Executor] ——— ExperienceRecord(success=True) ———→ [MemoryCurator]
[MemoryCurator] ——— 분석 후 Lesson 추출/정제 ———→ .agent/skills/, rules/
[MemoryCurator] ——— BrainStore (중요 지식) ———→ [BrainGateway]

```

이 구조에서 Planner와 Executor는 완전히 분리되어 있으며, 오로지 **Knowledge IR**이라는 계약(Contract)으로만 통신합니다.

이것은 마이크로서비스가 OpenAPI 스펙으로 통신하는 것과 정확히 동일한 패러다임입니다.

3. 상태 관리: .agent 폴더와 Brain의 정확한 대응

3.1 .agent 폴더 = Local Persistent DB + Config 저장소

```

.agent/
├── tools/           ← 서비스의 실행 가능한 모듈 (Executor의 도구)
├── rules/           ← 비즈니스 규칙 (Planner가 참고)

```

```
└── skills/          ← Planner/Executor의 기술 명세 (SKILL.md)
└── lessons/         ← MemoryCurator가 축적한 학습 로그 (appended)
└── memory/
    ├── working.json ← 현재 세션의 작업 메모리 (in-memory DB 역할)
    └── episodic.json ← 에피소드 기록 (history)
└── config.yaml      ← 환경 설정
```

이것은 전통적인 마이크로서비스의 **로컬 파일 기반 데이터베이스** (예: SQLite, RocksDB)와 동일한 역할을 합니다.

3.2 Brain = Distributed Shared Database / Knowledge Mesh

Brain은 **프로젝트 간, 에이전트 간 공유되는 전역 지식베이스**입니다.

예를 들어, 한 프로젝트에서 배운 "OKF에서 count(distinct student_id)는 반드시 project_participation.csv를 거쳐야 한다"는 Lesson을 다른 프로젝트에서도 재사용할 수 있게 해줍니다.

마이크로서비스 아키텍처의 **Redis Cluster, Elasticsearch, Apache Kafka**의 **영구 스토리지**처럼, Brain은 공유된 영구 저장소이면서도 실시간 쿼리가 가능한 **Knowledge Mesh**입니다.

4. Local Coding Agent를 이용한 PoC 구현 전략

우리의 CKOS를 **로컬 Coding Agent**(예: Ollama + Llama3, 또는 CodeQwen)와 함께 구축하는 PoC 계획입니다.

핵심은 각 **Agent**를 별도의 프로세스나 스레드로 구현하지 않고, 하나의 **Python 프로세스 내에서 메시지 큐와 모의 LLM 호출로 시뮬레이션**하는 것입니다.

이렇게 하면 아키텍처의 완전성을 검증하면서도 실제 분산 환경으로의 전환이 용이합니다.

4.1 구현 스택

- 언어: Python 3.11
- 메시지 버스: `asyncio.Queue` (경량), 또는 `Redis Pub/Sub` (확장 고려)
- LLM 호출: Ollama API (로컬), `llama-cpp-python` (오프라인), 또는 가짜 LLM (테스트 초기)
- 데이터: 샘플 OKF 번들 (CSV, index.md)

4.2 각 Agent의 구현 패턴

모든 Agent는 `BaseAgent`를 상속받고, `handle_message(msg)` 메서드를 구현합니다.

```
# base_agent.py
class BaseAgent:
    def __init__(self, name: str, message_bus: asyncio.Queue):
        self.name = name
        self.bus = message_bus
        self.inbox = asyncio.Queue()
        # 각 에이전트는 자신의 .agent 폴더 경로를 가짐
        self.local_store = Path(f".agent/{name}/")

    async def run(self):
        while True:
            msg = await self.inbox.get()
            response = await self.handle_message(msg)
            if response:
                await self.bus.put(response)

    async def handle_message(self, msg: AgentMessage) -> Optional[AgentMessage]:
        raise NotImplementedError
```

Planner Agent Skeleton

```
# planner_agent.py
class PlannerAgent(BaseAgent):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.llm = OllamaClient("llama3:8b") # 로컬 LLM
        self.skill = self.load_skill("skills/planner.md")

    async def handle_message(self, msg: AgentMessage) -> AgentMessage:
        if isinstance(msg, PlanRequest):
            # 1. 스키마 컨텍스트 로드 (OKF index.md에서)
            schema = self.load_okf_index(msg.question)
            # 2. LLM을 통해 KnowledgeIR 생성 (프롬프트: skill + schema + question)
            ir_json = await self.llm.generate(
                system=self.skill,
                prompt=f"Schema:\n{schema}\n\nQuestion: {msg.question}\n\nKnowledgeIR:"
            )
            ir = KnowledgeIR.parse_raw(ir_json)
            ir.sender = self.name
            ir.receiver = "Executor"
            return ir
        elif isinstance(msg, MemoryUpdate):
            # 새로운 Lesson이 있으면 로컬 스킬 업데이트
            self.apply_lessons(msg.new_lessons)
            return None
```

Executor Agent Skeleton

```
# executor_agent.py
class ExecutorAgent(BaseAgent):
    async def handle_message(self, msg: AgentMessage) -> AgentMessage:
        if isinstance(msg, KnowledgeIR):
            try:
                result_data = self.execute_ir(msg)
                # 결과를 ExecutionResult로 포장
                exec_result = ExecutionResult(
                    sender=self.name,
                    receiver="QueryRouter",
                    result=result_data,
                    citations=self.collect_citations(msg),
                    summary=self.summarize(result_data)
                )
                # MemoryCurator에게 경험 전달 (성공)
                experience = ExperienceRecord(
                    sender=self.name, receiver="MemoryCurator",
                    question=msg.question_context, plan=msg, success=True
                )
                await self.bus.put(experience)
                return exec_result
            except Exception as e:
                experience = ExperienceRecord(..., success=False, error_detail=str(e))
                await self.bus.put(experience)
                # 오류 결과 반환
                return ExecutionResult(..., result={"error": str(e)})
```

```
# memory_curator.py
class MemoryCuratorAgent(BaseAgent):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.lessons_file = self.local_store / "lessons/accepted.jsonl"

    async def handle_message(self, msg: AgentMessage) -> AgentMessage:
        if isinstance(msg, ExperienceRecord):
            new_lesson = await self.extract_lesson(msg)
            if new_lesson:
                # append to lessons file
                append_to_jsonl(self.lessons_file, new_lesson)
                # Planner에게 업데이트 알림
                update = MemoryUpdate(
                    sender=self.name, receiver="Planner",
                    new_lessons=[new_lesson]
                )
                return update
            return None
```

4.3 PoC 전체 Orchestrator

```
# orchestrator.py
import asyncio

async def main():
    bus = asyncio.Queue()
    router = QueryRouterAgent("Router", bus)
    planner = PlannerAgent("Planner", bus)
    executor = ExecutorAgent("Executor", bus)
    curator = MemoryCuratorAgent("Curator", bus)

    # 에이전트 등록 (라우팅 테이블)
    agents = {
        "Planner": planner.inbox,
        "Executor": executor.inbox,
        "MemoryCurator": curator.inbox,
        "QueryRouter": router.inbox,
    }

    # 디스패처: bus에서 메시지를 읽어 receiver의 inbox로 전달
    async def dispatcher():
        while True:
            msg = await bus.get()
            if msg.receiver in agents:
                await agents[msg.receiver].put(msg)

    # 각 에이전트 실행
    tasks = [
        asyncio.create_task(router.run()),
        asyncio.create_task(planner.run()),
        asyncio.create_task(executor.run()),
```

```

        asyncio.create_task(curator.run()),
        asyncio.create_task(dispatcher()),
    ]

    # 사용자 입력 시뮬레이션
    user_question = "2026년 1학기 프로젝트 참가 학생 수"
    await router.inbox.put(PlanRequest(sender="User", receiver="QueryRouter", question=user_question))

    await asyncio.gather(*tasks)

asyncio.run(main())

```

4.4 PoC 검증: TDD 평가 재활용

이 아키텍처의 가장 큰 장점은 **기존에 설계한 TDD 평가 체계를 그대로 사용할 수 있다**는 것입니다.

- `test_planner.py` 는 `Planner`의 `handle_message` 에 `PlanRequest` 를 보내고 반환된 `KnowledgeIR` 을 검증합니다.
- `test_executor.py` 는 `KnowledgeIR` 을 `Executor`에 보내고 `ExecutionResult` 를 검증합니다.
- 통합 테스트는 `Orchestrator`에 사용자 질문을 주입하고 최종 `ExecutionResult` 의 정확도와 인용을 평가합니다.

이것은 마이크로서비스의 `Contract Test`와 동일하며, **LLM/Agent Stack이 바뀌어도 테스트 케이스는 불변**입니다.

5. 결론: Micro-Agent Architecture = 진화된 Micro-Service

당신이 설계한 CKOS는 단순한 AI 챗봇이 아니라,
자연어를 입력받고, 계획을 수립하며, 데이터를 질의하고, 스스로 학습하는 지식 처리 마이크로서비스 시스템입니다.

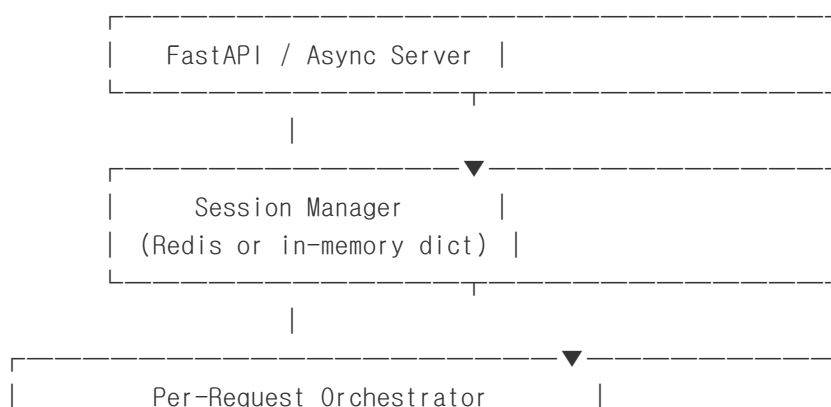
- **Message**는 JSON이며, 이는 `Knowledge IR`, `PlanRequest`, `ExperienceRecord`라는 도메인 언어입니다.
- **.agent** 폴더와 **Brain**은 각각 로컬 DB와 분산 공유 DB에 대응하며, `Agent`의 상태와 지식을 영속화합니다.
- **Local Coding Agent PoC**는 이 전체 아키텍처를 단일 프로세스에서 구현하고, 추후 gRPC나 NATS로 손쉽게 분산할 수 있는 발판입니다.

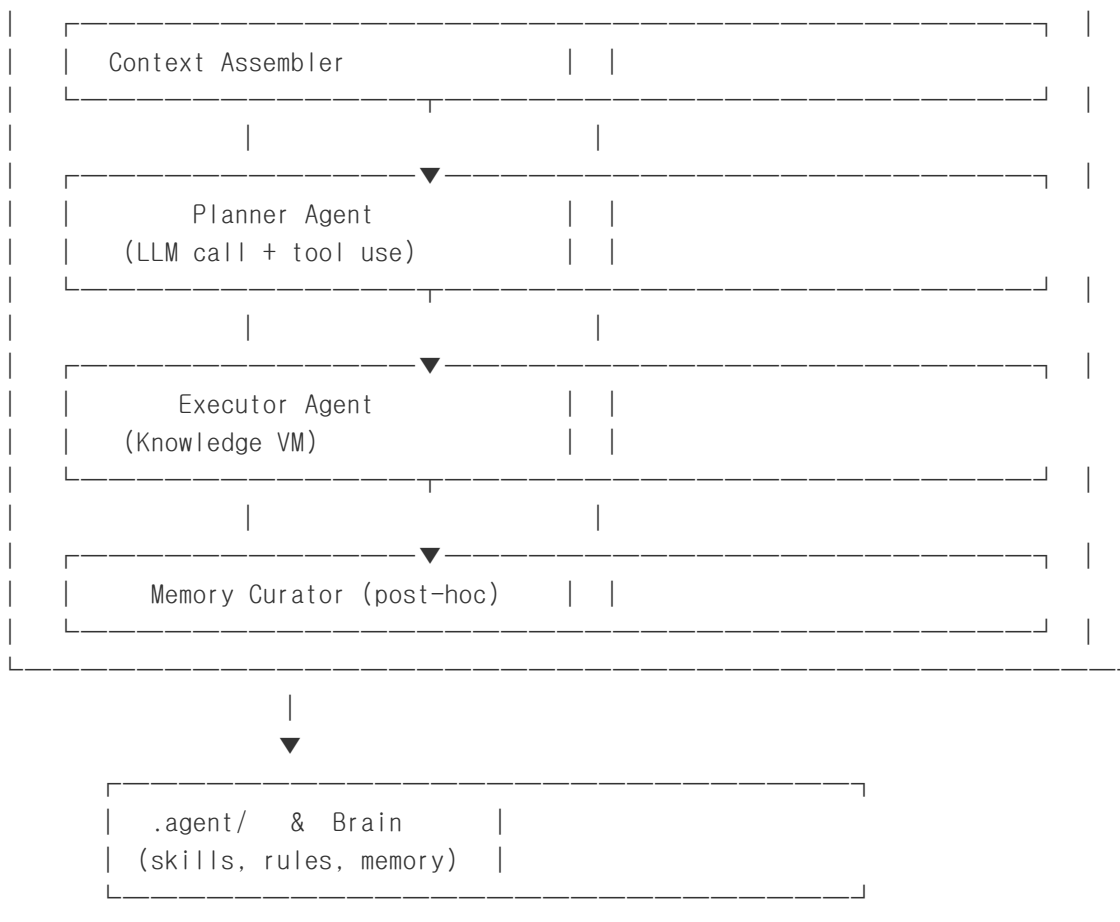
이제 이 PoC를 실제로 구현하면, 당신은 ****"LLM을 내장한 차세대 마이크로서비스 패러다임"**의 실증 사례를 가지게 될 것입니다. 이 구조 자체가 하나의 논문이 될 수 있습니다.

From Coding Agent to Dedicated Multi-User Agent Service

기존에는 Codex와 같은 외부 Coding Agent가 `.agent` 폴더의 내용을 **자체 규칙으로** 컨텍스트에 주입해 주었습니다. 이제 우리는 **전용 Agent Service**를 구축하면서, 그 컨텍스트 조립 로직을 **완전히 직접 제어**할 수 있게 됩니다. 아래에 `.agent` 및 `brain` 폴더는 그대로 유지하면서, 다중 사용자 도메인 특화 에이전트(DSA)를 구현하는 구체적인 방안을 제시합니다.

1. 전체 아키텍처 (Multi-User Service)





이 구조는 이전의 마이크로-에이전트 설계를 **서버리스/멀티유저**로 확장한 것입니다.

2. Session 관리 및 사용자별 상태 격리

각 사용자 요청은 `session_id`를 통해 구분되며, 모든 사용자별 상태는 **Session Store**에 저장됩니다.

- **Conversation History:** 사용자-어시스턴트 대화 로그 (최근 N턴)
- **Working Memory:** 현재 진행 중인 작업 맥락 (예: 탐색 중인 OKF 경로, 임시 결과)
- **User Preferences:** 응답 스타일, 권한 등

```

# session_manager.py
from typing import Dict, Optional
import uuid, time, json

class Session:
    def __init__(self, session_id: str):
        self.session_id = session_id
        self.history: list[dict] = []          # {"role": ..., "content": ...}
        self.working_memory: dict = {}        # 현재 작업 컨텍스트
        self.created_at = time.time()
        self.last_active = time.time()

class SessionManager:
    def __init__(self, storage: "Redis|dict"):
        self.store = storage

    async def get_or_create(self, session_id: str) -> Session:
        if session_id not in self.store:
            self.store[session_id] = Session(session_id)
        return self.store[session_id]
  
```

```

async def update_working_memory(self, session_id: str, update: dict):
    session = await self.get(session_id)
    session.working_memory.update(update)
    session.last_active = time.time()

async def append_history(self, session_id: str, role: str, content: str):
    session = await self.get(session_id)
    session.history.append({"role": role, "content": content})
    # Trim to last 20 turns
    if len(session.history) > 40:
        session.history = session.history[-40:]

```

핵심: Session은 사용자별로 격리되며, `.agent/` 아래의 공유 지식(스킬, 룰, 레슨)과는 분리됩니다.

3. Full Context Assembly: 주입되는 모든 것의 통제권

이전에는 Codex가 이 모든 것을 블랙박스로 처리했지만, 이제 우리는 **Context Assembler**를 통해 명시적으로 조립합니다.

3.1 조립 대상 목록

계층	내용	출처	적용 방식
System Prompt	에이전트 역할, 기본 규칙, 도구 목록	<code>.agent/AGENTS.md</code> + <code>SKILL.md</code> (공통)	매 호출마다 고정 prefix
Shared Rules	모든 에이전트가 따라야 할 규칙	<code>.agent/rules/*.md</code> (공통)	System prompt에 추가
Shared Lessons	교훈 (예: "DuckDB를 사용해라")	<code>.agent/lessons/accepted.jsonl</code> (공통)	동적으로 필터링하여 System prompt에 추가
User Working Memory	현재 작업 중인 임시 정보	Session의 <code>working_memory</code>	User message 앞에 추가하거나 Tool로 제공
Conversation History	최근 대화	Session의 <code>history</code>	Messages 배열에 포함 (role: user/assistant)
Brain Recall	외부 브레인 검색 결과	<code>brain_bridge.recall(query)</code>	도구 호출 결과로 추가하거나, Planner의 System prompt에 미리 삽입 (옵션)
Tool Definitions	사용 가능한 함수 목록	<code>.agent/tools/*.py</code> 의 서명	OpenAI function calling 형식으로 제공

3.2 Context Assembler 구현

```

# context_assembler.py
from pathlib import Path
import json

class ContextAssembler:
    def __init__(self, agent_dir: Path = Path(".agent")):
        self.agent_dir = agent_dir

```

```

self.system_prompt_base = self._load_system_prompt()
self.tool_definitions = self._load_tool_definitions()

def _load_system_prompt(self) -> str:
    # AGENTS.md + SKILL.md 합치기
    base = ""
    for md_file in ["AGENTS.md", "SKILL.md"]:
        path = self.agent_dir / md_file
        if path.exists():
            base += path.read_text() + "\n"
    return base

def _load_tool_definitions(self) -> list[dict]:
    # .agent/tools/*.py 에서 함수 시그니처를 읽어 JSON Schema로 변환 (생략)
    # 예시로 하드코딩
    return [
        {
            "name": "read_index",
            "description": "Read an index.md file from OKF bundle",
            "parameters": {
                "type": "object",
                "properties": {
                    "path": {"type": "string", "description": "Relative path within bundle"}
                },
                "required": ["path"]
            }
        },
        # ... 다른 도구들
    ]

def build_context(self,
                  session: "Session",
                  user_question: str,
                  extra_lessons: Optional[list[str]] = None,
                  brain_retrieved: Optional[str] = None) -> list[dict]:
    messages = []

    # 1. System Prompt
    system_content = self.system_prompt_base

    # Add shared rules
    rules_dir = self.agent_dir / "rules"
    for rule_file in sorted(rules_dir.glob("*.md")):
        system_content += f"\n\n[Rule] {rule_file.stem}:\n{rule_file.read_text()}"

    # Add relevant lessons (dynamic selection possible)
    lessons = self._get_relevant_lessons(session, extra_lessons)
    if lessons:
        system_content += "\n\n## Lessons Learned\n" + "\n".join(f"- {l}" for l in lessons)

    # Add brain recall if provided
    if brain_retrieved:
        system_content += f"\n\n## Relevant Knowledge from Brain\n{brain_retrieved}"

    messages.append({"role": "system", "content": system_content})

```

```

# 2. Conversation History
for msg in session.history[-20:]: # last 20 messages
    messages.append(msg)

# 3. User working memory (optional, injected as a user-like message)
wm = session.working_memory
if wm:
    wm_str = "## Working MemoryWn" + json.dumps(wm, ensure_ascii=False)
    messages.append({"role": "user", "content": wm_str})

# 4. Current user question
messages.append({"role": "user", "content": user_question})

return messages

def _get_relevant_lessons(self, session, extra: list[str] = None) -> list[str]:
    lessons_path = self.agent_dir / "lessons" / "accepted.jsonl"
    if not lessons_path.exists():
        return []
    # 간단히 모두 가져오거나, 임베딩 기반 선택 가능
    with open(lessons_path) as f:
        all_lessons = [json.loads(line)["content"] for line in f]
    # 추가 교훈이 있으면 합침
    if extra:
        all_lessons.extend(extra)
    # Token 제한을 고려하여 최대 5개만
    return all_lessons[-5:]

```

4. Agent Loop 구현: Per-User Orchestrator

이제 다중 사용자 요청을 처리하는 **Agent Loop**를 설계합니다.

이 루프는 각 사용자 요청에 대해 독립적으로 실행되며, Planner → Executor → 응답 생성의 순서를 따릅니다.

```

# agent_loop.py
import asyncio
from context_assembler import ContextAssembler
from session_manager import SessionManager
from planner_agent import PlannerAgent
from executor_agent import ExecutorAgent
from memory_curator import MemoryCurator

class MultiUserAgentService:
    def __init__(self):
        self.session_mgr = SessionManager(dict()) # 실제로는 Redis
        self.context_assembler = ContextAssembler()
        self.planner = PlannerAgent(self.context_assembler)
        self.executor = ExecutorAgent()
        self.memory_curator = MemoryCurator()

    async def handle_user_message(self, session_id: str, question: str) -> dict:
        # 1. 세션 로드
        session = await self.session_mgr.get_or_create(session_id)
        await self.session_mgr.append_history(session_id, "user", question)

```

```

# 2. Planner 호출 (LLM + 도구 사용)
#   Planner 내부에서 context_assembler.build_context() 사용
ir = await self.planner.plan(session, question)

# 3. 실행
result = await self.executor.execute(ir)

# 4. 응답 생성
answer_text = result.get("summary", "")
await self.session_mgr.append_history(session_id, "assistant", answer_text)

# 5. 작업 메모리 갱신 (Planner가 탐색한 경로 등)
if "working_memory_updates" in ir:
    await self.session_mgr.update_working_memory(session_id, ir["working_memory_update"])

# 6. 비동기로 Lesson 추출 (Memory Curator)
asyncio.create_task(
    self.memory_curator.record_experience(question, ir, result)
)

return {"answer": answer_text, "citations": result.get("citations", [])}

```

PlannerAgent의 구현 (LLM 호출 부분):

```

# planner_agent.py
class PlannerAgent:
    def __init__(self, context_assembler):
        self.assembler = context_assembler
        self.tools = self._init_tools() # read_index 등 실제 함수 매핑

    async def plan(self, session, question: str) -> dict:
        # 가능하면 Brain에서 관련 지식 검색
        brain_knowledge = await brain_bridge.recall(question) if brain_bridge else None

        # 컨텍스트 구축
        messages = self.assembler.build_context(
            session, question,
            brain_retrieved=brain_knowledge
        )

        # LLM 호출 (with tool use loop)
        llm_response = await self._call_llm(messages, self.tools)
        # LLM이 도구를 여러 번 호출한 후 최종 IR JSON을 반환한다고 가정
        ir = json.loads(llm_response.choices[0].message.content)
        return ir

    async def _call_llm(self, messages, tools):
        # OpenAI API 호출 예시 (비동기)
        import openai
        response = await openai.ChatCompletion.acreate(
            model="gpt-4o",
            messages=messages,
            tools=tools,
            tool_choice="auto"
        )

```

```
)  
# Tool call loop 구현 (생략)  
return response
```

5. 동시성 및 확장성 고려

- **FastAPI + asyncio**를 사용하면 각 요청이 별도의 비동기 작업으로 처리됩니다.
- `SessionManager`는 `Redis`를 백엔드로 사용하여 여러 프로세스/서버에서 상태를 공유할 수 있습니다.
- `.agent/` 폴더의 파일은 읽기 전용으로 공유하지만, `lessons`는 쓰기가 발생하므로 **락** 또는 **단일 writer** 패턴을 적용합니다. (예: `Memory Curator`만 쓰기)

```
# main.py (FastAPI app)  
from fastapi import FastAPI, HTTPException  
app = FastAPI()  
service = MultiUserService()  
  
@app.post("/ask")  
async def ask(session_id: str, question: str):  
    try:  
        answer = await service.handle_user_message(session_id, question)  
        return answer  
    except Exception as e:  
        raise HTTPException(status_code=500, detail=str(e))
```

6. `.agent` + `brain` 폴더 유지하며 전환하는 핵심 이점

- 기존에 `Codex`와 함께 작업하며 쌓아둔 **Skills, Rules, Lessons**는 그대로 재사용됩니다.
- `Brain`은 **프로젝트 간 지식 공유**를 가능하게 하여, 다른 대학이나 부서에서도 공통 `Lesson`을 활용할 수 있습니다.
- 이제 우리는 **컨텍스트에 무엇을 주입할지 완전히 결정**할 수 있기 때문에, 불필요한 정보로 인한 환각이나 토큰 낭비를 방지할 수 있습니다.
- TDD 평가 프레임워크는 `session_id`를 부여한 상태에서 `service.handle_user_message`를 호출하는 것으로 그대로 작동합니다.

7. 결론: Agent OS로의 완전한 이행

당신은 이제 단순한 코딩 도우미를 넘어,

사용자 세션과 공유 지식을 관리하는 지식 처리 운영체제를 갖게 됩니다.

`.agent`와 `brain`은 그 OS의 파일 시스템과 분산 메모리이며,

Context Assembler는 그 OS의 프로세스별 주소 공간을 구성하는 커널의 메모리 관리자 역할을 합니다.

이 설계를 그대로 구현하면,

“자연어로 질문하고, 지식을 자동 탐색하여 검증된 답변을 생성하며, 경험을 통해 스스로 진화하는 다중 사용자 서비스”를 완성할 수 있습니다.